DataSentry AI: A Paired Evaluation of Training-Data-Guided Cleaning for Tabular Regression

Tilaksinh Chauhan (23000740), Kavya Jariwala (23000543),
Bhargav Panchal (23000502), Om Nathani (23000883)
School of Engineering and Technology
Navrachana University
India

Abstract—Unreliable tabular data can alter predictive performance, but a quality warning does not show whether a particular repair helps. We present DataSentry AI, a local CSV profiling application and a paired regression benchmark that tests a transparent training-guided cleaning policy against three fixed recipes. Three public UCI tasks—early student-grade prediction, red-wine quality, and building heating load—are evaluated under random missingness, feature-dependent missingness, and synthetic numeric extremes at 0%, 5%, 15%, and 30% of eligible training-feature cells. Five grouped 80/20 splits, four strategies, and three models yield 2,160 fitted arms. Each comparison uses the same split and synthetic corruption mask; imputers, encoders, clipping bounds, and the policy decision are fit from training rows only. Held-out rows remain free of synthetic corruption. The policy improves mean held-out root mean squared error (RMSE) over fixed median/mode in 31 of 81 positive-corruption dataset–mechanism–level–model conditions, ties it in 36, and worsens it in 14. At 15% numeric extremes, the policy reduces paired RMSE for Ridge and a neural multilayer perceptron on the energy task, but raises it for the forest on student data and the neural model on wine data. These findings support a context-dependent effect, not a universal advantage for adaptive cleaning. We provide source checksums, run-level scores, split and mask hashes, and reproducible figures. The study's scope is controlled training-data corruption on three small regression tasks; natural errors and polluted serving data require separate evaluation.

Index Terms—data quality, tabular regression, missing values, data cleaning, model evaluation, data leakage

I. Introduction

Machine-learning systems rely on data collected for purposes that may differ from the eventual prediction task. An apparently ordinary CSV file can contain empty cells, duplicate rows, implausible numerical measurements, or categories that represent the same concept with different strings. Production validation systems make such problems visible, but identifying a suspicious value and proving that a particular repair improves predictive utility are distinct tasks [1], [2]. A row can be unusual yet valid; a duplicate can be a legitimate repeated observation; a missing value can carry information about how the record was produced. Cleaning without considering these meanings may remove signal.

The interaction between data quality and prediction has been studied directly. CleanML compared cleaning across real classification datasets and several error types [3]. ActiveClean prioritized cleaning records that matter to a model, and BoostClean used predictive feedback to combine repair operations [4], [5]. AutoClean evaluated cleaning as part of an AutoML search and found that more elaborate treatments did not always improve downstream performance [6]. Conformal Data Cleaning and HyperImpute provide more capable automated repair or imputation schemes [7], [8]. A broad 2025 study compared the impact of multiple corruption dimensions and the location of pollution in training or testing [9]. These results rule out a general claim that cleaning is always useful or that data-dependent selection itself is new.

The remaining practical question is narrower. On modest tabular regression datasets, is a small, interpretable rule based on observed training characteristics useful enough to outperform one fixed recipe, and under which conditions does it fail? Such a rule is inexpensive to inspect, but has less flexibility than a search system. We measure that trade-off rather than presume a benefit. We explicitly separate the performance of a minimally repaired model trained on a polluted table from a model trained after optional treatments. Every arm still performs whatever encoding or imputation its estimator requires; "raw" cannot mean passing missing strings directly to an estimator that cannot accept them.

Our first illustrative task uses the Portuguese-course Student Performance dataset [10], [11]. The target is the final grade, $G3$. The source documentation notes the strong relation of earlier period grades $G1$ and $G2$ to $G3$; a first demonstration that asks what can be learned before those grades exist must exclude them. We also exclude absences in the early-prediction version because total absences may not be known at the proposed decision time. This is a feature-availability decision, not merely a statistical feature-selection choice. A warning about such leakage is supported by the broader data-mining literature [12]. We use this small educational dataset as a demonstration, not as evidence that a cleaning policy will

generalize to all educational or tabular tasks.

Two additional tasks add variation that one student table cannot provide. Physicochemical measurements are used to predict an ordinal red-wine quality score, which we deliberately treat as a numeric regression response for this comparison. Building attributes from a simulated energy-efficiency design predict heating load. These targets have different units, feature distributions, and duplicate structures. Consequently we never average their raw RMSE values into one headline number. Instead, the policy is compared with a fixed recipe on matched splits within each task, then its direction and relative magnitude are summarized across conditions. This design follows the general benchmarking principle that task definition and evaluation rules must be explicit before scores are compared [13].

The synthetic intervention is intentionally narrow. We damage only observed training-feature cells, leaving target labels and held-out records unchanged. This isolates the effect of training-data quality under a clean evaluation set. It does not estimate deployment performance when future inputs are also polluted; that would be a distinct question, as corruption-benchmark work emphasizes [9], [14]. Nor do synthetic missingness and extreme values establish that every naturally missing or unusual cell should be altered. Real collection processes may encode meaningful absence or valid rare cases. Our language therefore distinguishes a flag, a known synthetic change, and a justified repair.

The implemented study has four parts: a descriptive quality profile that distinguishes observations from verified errors; a deterministic training-feature corruption generator; fixed and training-guided preprocessing arms; and a paired held-out comparison. We examine three UCI regression tasks and preserve the seeds, dataset hashes, split and corruption-mask hashes, chosen operations, and individual scores. A Streamlit interface allows a local CSV to be profiled and the comparison to be run without changing the experimental rules. The final result is deliberately mixed: the simple policy wins in some conditions, loses in others, and often selects the fixed baseline. This is an empirical boundary for an interpretable heuristic, not evidence of a new general-purpose repair method.

II. Literature Review

A. Data quality, validation, and repair

Data quality has several dimensions, and their effects depend on the downstream task. The 2025 empirical study by Mohammed et al. examined completeness, consistency, feature accuracy, uniqueness, and other dimensions over classification, regression, and clustering. It distinguished corrupted training, corrupted testing, and both [9]. JENGA likewise provides a framework for injecting errors to test model predictions, emphasizing that an issue in serving data can have a different consequence from an issue in training data [14]. These studies motivate our decision to hold test features clean in the primary training-quality experiment and to label a later serving-quality stress test separately.

The dimensions are related but not interchangeable. A missing value is directly countable once the representation of blanks is defined; an exact duplicate depends on which columns define a record; an "inconsistent" category depends on a normalization rule or domain vocabulary; and an outlier depends on a reference distribution. A data-validation rule may catch a schema violation without specifying its correct replacement. Breck et al. locate validation in the larger production ML workflow, where an input anomaly can affect subsequent training and serving stages [1]. That perspective argues for preserving the raw input and documenting the rule that produced every flag. We use exact counts and named thresholds in the interface, and make no confidence or causal claim from a flagged cell alone.

Tools that detect or repair questionable entries solve related but different problems. Breck et al. describe validation of input data to ML systems; the Data Quality Toolkit combines assessment and remediation suggestions [1], [15]. Raha learns to find erroneous cells from limited labels, while HoloClean combines constraints and statistical signals in a probabilistic repair model [2], [16]. PClean encodes domain knowledge in a probabilistic program; DataAssist offers an integrated preparation workflow [17], [18]. None of these mechanisms makes a domain-free outlier flag equivalent to ground truth. They also imply different assumptions about annotations, constraints, or usable domain knowledge. The DataSentry report therefore uses the terms missing, exact duplicate, pattern inconsistency, and potential outlier precisely. Its optional category repair is limited to deterministic text normalization.

These repair systems demonstrate why a single notion of "clean data" is inadequate. Constraint-driven repair can be effective when a field is governed by a known relationship, and annotation-based detection can prioritize human attention when labels for errors exist [2], [16], [17]. A small uploaded CSV may offer neither. Our conservative string-pattern flag groups values only when Unicode compatibility normalization, trimming, whitespace collapse, and case folding coincide. Even then, the profile reports the variants and counts rather than silently rewriting them. The optional normalization arm in the application is kept separate from the four-recipe paper experiment because a domain's capitalization or spacing may itself be meaningful. Likewise, duplicate removal is an optional analysis, while grouped splitting prevents exact feature copies from appearing on both sides of an evaluation split without presuming that all duplicates are wrong.

The effect of cleaning on downstream accuracy is an established research topic. ActiveClean models cleaning as a progressive, task-directed process, and BoostClean automatically searches combinations of detectors and

repairs [4], [5]. CleanML systematically compared error types, models, and cleaning algorithms for classification [3]. AutoClean embedded imputation and outlier operations within a broader AutoML search; it found that model choice can compensate for some errors and that expensive cleaning may not pay for itself [6]. Conformal Data Cleaning uses calibrated predictions for cell-level repair and reports gains in a majority of its evaluated cases [7]. Picket instead seeks to flag corrupted examples at both training and inference time, and a robust variational autoencoder models mixed-type cell outliers [19], [20]. Our question is different in scale: compare an explicitly written, low-compute rule to fixed recipes while preserving exact split parity and reporting its failures. We do not claim that such a rule is the first data-driven cleaning method.

ActiveClean and BoostClean illustrate a useful distinction between repairing the most consequential records and running an untargeted transformation over every flagged value [4], [5]. AutoClean and Conformal Data Cleaning extend this task-aware perspective with different search or uncertainty mechanisms [6], [7]. We do not reproduce their optimization budgets, annotations, or repair models. Our contribution is instead a fully specified small-policy comparator whose decisions can be printed for every run. This permits a clear negative result: if a policy selects the same fixed recipe for many conditions, it has supplied no extra benefit, even though it is called data-driven. A broad empirical comparison such as Mohammed et al.'s strengthens the case for reporting by corruption mechanism and model rather than collapsing all runs into a favorable aggregate [9].

B. Missing-data mechanisms and imputation

Missingness is a mechanism, not only a fraction. Rubin's formulation distinguishes conditions under which the missing-data process may be ignored for inference, and later methodological work cautions against treating all missingness as interchangeable [21], [22]. Our synthetic random masking is an MCAR-like stress test, while feature-dependent masking approximates a MAR-like situation. Neither synthetic scheme proves what mechanism generated naturally missing entries. It is therefore important to keep the two schemes separate in figures and tables and to avoid a blanket statement such as "15% missing data" without specifying how cells were selected.

In the random mechanism, every eligible observed training cell is sampled without replacement with equal probability. In the feature-dependent mechanism, one observed training feature is retained as a driver, and the chance of masking cells in other columns increases with the driver's rank. This is a controlled construction rather than an estimate of a real missingness process. It yields a known set of affected cells and a reproducible mask for method comparison. Because the driver stays observed, the mechanism fits the intuition of missingness associated with available information. It does not cover missing-not-at-random behavior, and the choice of a highest-variance numeric driver is scale-sensitive. These restrictions matter when interpreting any advantage of a KNN or fixed imputer [21], [22].

Median or mode imputation is simple and deterministic, but it ignores relations among columns. The early KNN imputation study by Troyanskaya et al. used local similarity to estimate missing cells in microarray data [23]. MICE iteratively specifies conditional models for incomplete variables, and missForest uses random forests for mixed-type imputation [24], [25]. HyperImpute demonstrates an adaptive iterative method with automatic model selection [8]. These approaches show why a richer imputer can be useful, but they also add fitting cost and assumptions. The method that best reconstructs masked cells need not be the method that best preserves held-out target prediction. Our primary metric is therefore downstream RMSE, with cell reconstruction error reserved as a secondary diagnostic when synthetic ground truth is available.

KNN distance depends on how numeric features are scaled; a feature with a large numerical range can otherwise dominate the neighbor calculation. We place a scaler and the KNN imputer in the training-fitted numeric pipeline, while categorical values use training-derived modes and one-hot encoding. This comparison tests a specific, auditable KNN recipe rather than every possible distance metric or imputation model. More sophisticated chained-equation, forest, and adaptive imputers remain relevant alternatives [8], [24], [25]. Their existence also prevents us from interpreting a KNN win as proof that the proposed policy is close to the best possible cleaning procedure. The cost of fitting each recipe is recorded, since an error reduction can be too small to justify its computational expense in a practical pipeline [6].

C. Outliers, model choice, and evaluation validity

Anomaly detection methods capture different meanings of unusualness. LOF compares local density, Isolation Forest isolates atypical observations by random partitioning, and the anomaly-detection survey explains the diversity of detector assumptions [26]–[28]. A mixed-type robust variational autoencoder addresses cell-level detection and repair [20]. We begin with a simpler IQR-based flag and clipping baseline, explicitly calling the flag potential rather than verified corruption. Controlled numeric perturbations are useful because the affected cells are known; natural tail values are retained as a separate question. A treatment that clips a legitimate extreme may worsen prediction.

The IQR rule uses quartiles and a fixed 1.5 multiplier to define fences. It is transparent and computationally small, yet a narrow discrete distribution can flag a valid ordinal value, and a long-tailed continuous distribution can contain legitimate measurements outside its fences. LOF and Isolation Forest use different geometry and therefore need not identify the same observations [26], [27]. Our synthetic extreme generator changes a known fraction of observed

numeric training cells to values far from that column's training median. This enables a controlled response test, but it does not make IQR clipping universally justified. In particular, the student and energy tables contain integer-coded or designed numeric predictors; the intervention and the cap can affect these differently from a naturally continuous sensor measurement.

Model family is a confounder if a cleaning comparison silently changes the estimator. Random forests provide one strong classical tabular reference [29]. Neural architectures for tabular data have been studied in detail, but comparative benchmarks find that neural and tree methods trade places across dataset properties, sample sizes, and tuning budgets [30]–[32]. We therefore compare cleaning within each prespecified model before discussing differences among models. A multilayer perceptron serves as the course-required neural comparison; its presence does not make it the expected winner on a small dataset.

This is also why the paper does not rank cleaning strategies using a different winning model in each arm. A model-selection procedure that sees an outer held-out score would give the cleaning operation credit for an unrelated search decision. Ridge, forest, and multilayer perceptron receive fixed settings and the same transformed training condition within each comparison. The forest can be insensitive to monotonic rescaling but still react to missing-value repair or clipping; a linear model may respond strongly to extreme magnitudes; and a neural model has its own fitting variability. These are testable model-specific patterns, not assumptions that one family must always be robust [29]–[31].

Reproducible benchmarking requires more than reporting a top score. OpenML benchmarking suites argue for curated tasks and standardized evaluation [13]. Kaufman et al. formalized several forms of leakage, including information unavailable at prediction time [12]. In the present study, computing an imputation statistic, clipping bound, feature selection, or policy choice on the whole dataset before splitting would give the test set influence over model construction. The analysis code uses pandas for data handling and scikit-learn pipelines for the fitted transformations and estimators [33], [34]. These software references describe implementation, while the split and fit rules are part of the experimental protocol.

We use paired outer splits and shared corruption masks because otherwise a small apparent cleaning gain might come from different held-out observations or a different synthetic draw. Grouping identical feature records before splitting is especially relevant for the wine table, which contains many exact duplicate rows. The grouping key is formed from the permitted predictors, so repeated feature vectors cannot appear in both partitions even if their labels differ. Training-fitted transformations are then encapsulated in a pipeline. The held-out responses are used only to compute final metrics; the simple policy reads no labels or held-out statistics. This operationalizes the leakage concerns in Kaufman et al. and keeps the comparison interpretable [12], [34].

III. Research Direction

The question is: Across three modest tabular regression tasks, does a simple policy that selects a cleaning recipe from the current training features lower held-out RMSE relative to a fixed median/mode recipe under controlled missingness and numeric extremes? This is a targeted empirical comparison, not a claim that adaptive cleaning is new. The reference works in Table I already establish task-aware and automated repair. Our specific object of study is whether a frozen, easily auditable choice among inexpensive recipes offers a worthwhile benefit when the user can see the full decision and its failures.

H1 predicts that larger training-feature corruption generally increases held-out error, while allowing nonmonotonic individual runs. H2 predicts that the policy beats fixed median/mode in some matched conditions, but not every dataset, model, or mechanism. H3 predicts that IQR clipping can harm a model when flagged extremes are valid or the cap distorts informative predictors. The study was designed so a tie or loss counts as evidence against an unqualified policy benefit. All three hypotheses are examined through per-dataset effects rather than one pooled raw RMSE.

IV. Proposed Model and Experiment Setup

A. Application boundary and descriptive quality report

The implemented Python application accepts a CSV with a user-selected numeric response. It displays null and whitespace-only cell counts by column; exact duplicate full-row counts; potential numeric outliers from the 1.5-IQR rule; and categorical values that become identical under Unicode NFKC normalization, trimming, whitespace collapse, and case folding. Every finding has a name, count, and caveat. The original table remains available for inspection. The report is descriptive: its whole-table quartiles are for display and are never reused as model cleaning parameters. Train/test-sensitive cleaning statistics are fitted afresh inside the evaluation pipeline. Optional duplicate-removal and deterministic text-normalization arms are available in the interface as exploratory checks and are excluded from the prespecified four-strategy benchmark.

Figure 1 shows the boundary. A command-line runner and the interface call the same evaluation function; the interface can select factors but does not change splitting, corruption, or fitting semantics. Result rows include dataset and condition identifiers, selected strategy, scores, runtime, the number of affected cells, and hashes of the split and corruption mask. This makes the comparison auditable beyond a dashboard screenshot.

TABLE I
Selected prior work and the specific distinction of the present study. Scope descriptions summarize the cited papers; a blank cell is not a claim that a paper lacks other capabilities.

Study	Relevant contribution	Why it does not answer this paper's narrow question directly
CleanML [3]	Classification benchmark across errors and repairs	Does not establish the utility of this small transparent policy for regression
AutoClean [6]	Cleaning operations inside AutoML search	Selection is broader and more computationally involved
HyperImpute [8]	Adaptive iterative imputation	Focuses on a richer imputation procedure, not the present fixed-policy comparison
Conformal cleaning [7]	Automated cell-error cleaning with calibration	Evaluates a different repair mechanism
Mohammed et al. [9]	Broad corruption effects across tasks and data locations	Studies impact of quality dimensions rather than this policy's choice among simple repairs

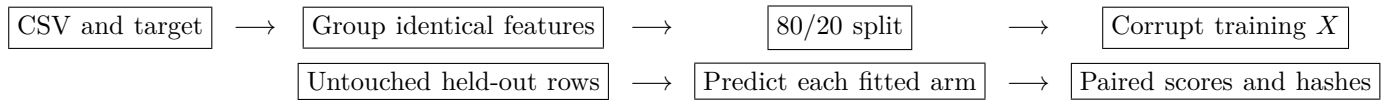


Fig. 1. Evaluation flow. Every cleaning decision and fitted transformation uses training rows. All arms within a condition share the split and synthetic corruption mask. The held-out partition receives no synthetic corruption.

TABLE II
Verified study datasets and baseline quality findings. Duplicate counts include full rows beyond the first occurrence. Target scales differ and are not pooled as raw RMSE.

Task	Rows	Predictors	Missing	Duplicates
Student $G3$	649	29	0	0
Red wine quality	1599	11	0	240
Energy heating	768	8	0	0

B. Datasets and feature availability

Table II gives the three public UCI tasks and the exact modeling boundaries. The committed comma-separated student table was compared by parsed values with the official semicolon-separated Portuguese-course source: all 649 rows, 33 columns, their order, and values matched. We predict $G3$ using 29 features, excluding $G1$, $G2$, and total absences for the stated early-prediction point. The red-wine table has 11 physicochemical predictors; its integer quality score is treated as a numeric regression outcome. The energy table retains eight building predictors and heating load, while the other response, cooling load, is excluded. These choices prevent a second target or later-time measurements from leaking into the predictors [10]–[12].

The three datasets vary in sample count, target scale, feature types, and repeated records. Wine's 240 exact full-row duplicates make naive row-level random splitting especially vulnerable to repeated information across train and test. The energy observations are a designed simulation grid, so unusual parameter combinations are not automatically invalid. The student features are a mixture of demographic, school, and ordinal coded fields. We use these differences to test transfer of a small cleaning rule, while recognizing that three tasks cannot establish population-wide superiority. Official source URLs, license information, original file hashes, normalized local copies, and regeneration code accompany the experiment.

C. Synthetic corruption and shared controls

For each dataset, seeds $\{11, 23, 37, 53, 71\}$ define five 80/20 GroupShuffleSplit partitions. The group key is the complete permitted feature vector after fixed blank-string normalization; identical feature rows stay on one side. Test rows are neither fitted on nor synthetically damaged. The three training-feature interventions are: (i) uniform masking without replacement over eligible observed cells (MCAR-like); (ii) masking of non-driver cells with probabilities increasing with an observed training-feature rank (feature-dependent, MAR-like); and (iii) replacement of selected observed numeric cells by the column training median plus or minus ten robust scale units. Rates are $\{0, 0.05, 0.15, 0.30\}$ of eligible cells. The response is never corrupted. The feature-dependent driver is kept observed; the mask is generated before any strategy or model is fit.

The same split and mask are reused for all four strategies and three models in a condition. A stored mask SHA-256 and test-index hash make accidental divergence detectable. The five seeds are repeated evaluations of each dataset, not five independent datasets. The primary design intentionally measures the consequences of unreliable training data under a clean held-out test. It does not silently combine that question with serving-time corruption, for which different conclusions could follow [9], [14]. Synthetic changed cells are known perturbations; naturally extreme source values are only potential errors.

D. Fixed recipes and frozen policy

The median recipe fills numeric gaps with training medians and categorical gaps with training modes; it

performs model-required encoding and scaling. KNN replaces the numeric median with a five-neighbor imputer after training-fitted scaling, while retaining categorical modes. IQR adds a training-fitted 1.5-IQR numeric cap after median imputation. The fourth arm chooses one of these three recipes by a rule frozen before the full experiment: choose KNN if the observed training-feature missing fraction is between 10% and 25%, there are at least 400 training rows, and at least two numeric columns; otherwise choose IQR if more than 5% of observed numeric cells exceed their training IQR fences; otherwise choose median/mode. The selector sees training features only. These candidates reflect simple imputation and clipping choices studied in richer forms elsewhere [6]–[8], [23], [25].

At 0% corruption, the median arm is the uncorrupted reference with the minimum processing required by the estimator. At positive missingness it is the minimally repaired corrupted baseline. A literally untreated missing-value table cannot be supplied to estimators that reject nulls; presenting such a failed fit as a valid "original" score would be misleading. The contrast therefore measures optional cleaning beyond a fixed, valid preprocessing recipe. The IQR arm may also alter genuine source extremes, and the policy may choose that arm in an uncorrupted condition. These effects are retained rather than filtered out.

E. Metrics and paired analysis

For held-out responses y_i and predictions $\hat{y}_i$ over n test rows, the primary metric is

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}. \tag{1}$$

Within dataset d, mechanism c, level l, seed s, and model m, we subtract the fixed median/mode RMSE from the alternative's RMSE:

$$\Delta_{d,c,l,s,m,r} = \mathrm{RMSE}_{d,c,l,s,m,r} - \mathrm{RMSE}_{d,c,l,s,m,\mathrm{median}}. \tag{2}$$

A negative Δ favors recipe r. The heatmap uses the mean of $100\Delta/\mathrm{RMSE}_{\mathrm{median}}$ over the five paired seeds, allowing visual comparison of directions across target scales. We also save held-out mean absolute error (MAE), R^2, model-fit runtime, affected-cell count, and the selected policy action. Figures show mean values and, where indicated, standard errors across the five splits. We report effects by dataset and model; no significance test treats the 2,160 fitted arms as independent samples.

V. Model Implementation

A. Leakage-safe processing and fixed estimators

Pandas reads the CSV, and scikit-learn pipelines encapsulate fitted transformations. After grouping and splitting, the corruption generator copies only the training features. A column transformer fits numeric imputation and scaling and categorical mode imputation with one-hot encoding on the training partition. Unseen held-out categories are ignored by the encoder. A wholly missing categorical training column receives a fixed sentinel rather than looking at held-out rows. The IQR cap is learned from the imputed training numeric matrix. The policy's missing fraction and IQR rate are computed from the corrupted training features; no target or held-out statistic is passed to it.

Ridge regression uses $\alpha = 1$; random forest uses 100 trees, a minimum leaf size of three, and a seeded estimator; the neural comparison is a one-hidden-layer, 32-unit multilayer perceptron with early stopping and at most 250 optimization iterations. Model settings are fixed across arms rather than tuned against the outer test set. Ridge, forest, and multilayer perceptron are compared within each condition, not selected to maximize a reported held-out score. The neural fits can reach the iteration limit without convergence; those warnings and the fixed small architecture limit what can be inferred about neural methods generally. The design still meets the requirement to evaluate a neural comparator without granting it a larger search budget than the classical methods.

B. Software workflow and verification

The application exposes the quality profile, CSV upload, target and feature-exclusion controls, experiment factors, paired score table, chart, selected operations, and CSV export. A separate command-line runner executes the full prespecified grid; the Streamlit app is a presentation layer around the same experiment function. Focused tests cover duplicate grouping, immutable source data, repeatable masks, an always-observed feature-dependent driver, training-only clipping, policy behavior, missing string normalization, mode fallback, and shared split/mask hashes. A local end-to-end smoke run produced the expected 24 quick-grid rows, and the interactive student demo completed 32 fits.

The complete run used Python 3.13, pandas 3.1.0, NumPy 2.5.4, scikit-learn 1.9.1, and matplotlib 3.11.2 on the user's local machine. Each of the three default runners wrote 720 rows and a manifest with the input SHA-256 and factor configuration. The 2,160 rows were validated before plotting: every seed and level appeared, each condition contained all four strategies, and strategies within a condition shared split and mask hashes. The plotting script reads these frozen CSVs and does not refit a model. Checksums, code, source data, result CSVs, figures, and setup instructions are provided with the project so the paper's numbers can be audited.

VI. Results and Discussion

A. Baseline sensitivity to corruption

Table III reports the median/mode arm at 0% and 30% random training-feature masking. All three student models worsen on average, but the increase is small on its grade-point scale. Energy heating load rises substantially,

TABLE III
Mean held-out RMSE for the fixed median/mode recipe, averaged over five grouped splits, before and after 30% random training-feature masking. Units belong to each target and must not be pooled.

Task	Model	0%	30%
Student	Ridge	2.805	2.897
	Forest	2.783	2.842
	MLP	2.765	2.869
Wine	Ridge	0.660	0.669
	Forest	0.643	0.667
	MLP	0.860	0.866
Energy	Ridge	2.957	4.100
	Forest	0.636	1.040
	MLP	5.372	6.455

especially for the forest and Ridge. Wine changes less, and its neural model is nonmonotonic across intermediate levels. Figure 2 shows the full four-level Ridge curves with standard-error bars. The figure also shows that IQR clipping can be worse than the no-cap recipe even without synthetic extremes, notably on student data; source-tail values are not necessarily erroneous. These are means over five grouped splits, not guarantees of monotonic harm for every split.

B. Where the training-guided rule helps and fails

Across the 81 positive-corruption dataset–mechanism–level–model condition means, the frozen policy has lower RMSE than median/mode in 31, exactly ties it in 36, and has higher RMSE in 14. Many ties occur because the policy deliberately returns the median recipe outside its thresholds. Being called "adaptive" therefore does not imply that a different treatment was applied. Figure 3 shows the paired percent effect at 15% corruption, where the selector makes the most distinct choices. Its signs differ even within one dataset: under synthetic numeric extremes, energy Ridge and MLP improve strongly, whereas student forest and wine MLP worsen. The forest on energy changes little.

Table IV gives absolute paired differences and between-seed standard deviations for the 15% numeric-extreme condition. On energy, Ridge falls from 9.728 to 5.583 RMSE units and the MLP from 13.401 to 8.119; their paired mean reductions are 4.145 and 5.282 units. On wine, the policy improves Ridge by 0.092 but worsens MLP by 0.227; on student data, it worsens the forest by 0.219. These contrasts were measured on the same held-out rows and corruption masks for each seed. The MLP's convergence warnings and weak uncorrupted performance on some tasks make its outcomes particularly sensitive to the fixed training budget. We therefore report them as one model comparison, not as a conclusion about neural networks in general.

The policy is the best or tied-best of the three fixed candidates in only 36 of the 81 positive-corruption condition means. It does not solve model or recipe selection in

TABLE IV
Policy minus median/mode held-out RMSE at 15% synthetic numeric extremes: mean paired difference Δ and standard deviation across five splits. Negative values favor the policy.

Task	Model	Mean Δ	SD
Student	Ridge	+0.010	0.030
	Forest	+0.219	0.093
	MLP	−0.129	0.199
Wine	Ridge	−0.092	0.011
	Forest	−0.000	0.002
	MLP	+0.227	0.075
Energy	Ridge	−4.145	0.207
	Forest	+0.004	0.005
	MLP	−5.282	0.125

TABLE V
Training-guided recipe selected at every positive-corruption condition. Each entry held across three datasets and five grouped splits; the policy decision is shared by the three models.

Mechanism	5%	15%	30%
Random missing	Median	KNN	Median
Feature-dependent	Median	KNN	Median
Numeric extremes	IQR	IQR	IQR

general. Its strong energy gains are partly a response to deliberately large synthetic extremes, while the harms on other tasks show that clipping can remove signal or interact poorly with a model. Random and feature-dependent missingness also differ: at 30% the policy usually reverts to median/mode and hence contributes no selection gain; at 15% it often chooses KNN. The rule's thresholds are transparent but crude, and their interpretation depends on the fraction of all features missing rather than the importance of particular columns.

C. What the policy actually selected

The decision log reveals an important limit that a mean score alone would obscure. Table V shows the selected recipe for each positive-corruption mechanism and level. Within each row, the choice was identical for all three datasets and all five splits. Numeric extremes triggered the IQR cap at every tested level; both missingness mechanisms triggered KNN at 15% and median/mode at 5% and 30%. Thus the simple rule used training-feature calculations, but in this experimental grid it behaved almost like a schedule indexed by corruption type and level. It did not discover a distinct task-specific recipe. This is one reason to avoid describing the 31 wins over median as evidence that the selector itself learned a broadly superior relationship between dataset characteristics and cleaning. The same gains could be produced by preselecting the fixed recipe for that condition if its mechanism were known in advance.

This selection pattern also clarifies the large number of exact policy ties. When the policy returns median/mode, the same fitted pipeline and model receive the same training data and random seed, so a tie is a real repeated recipe

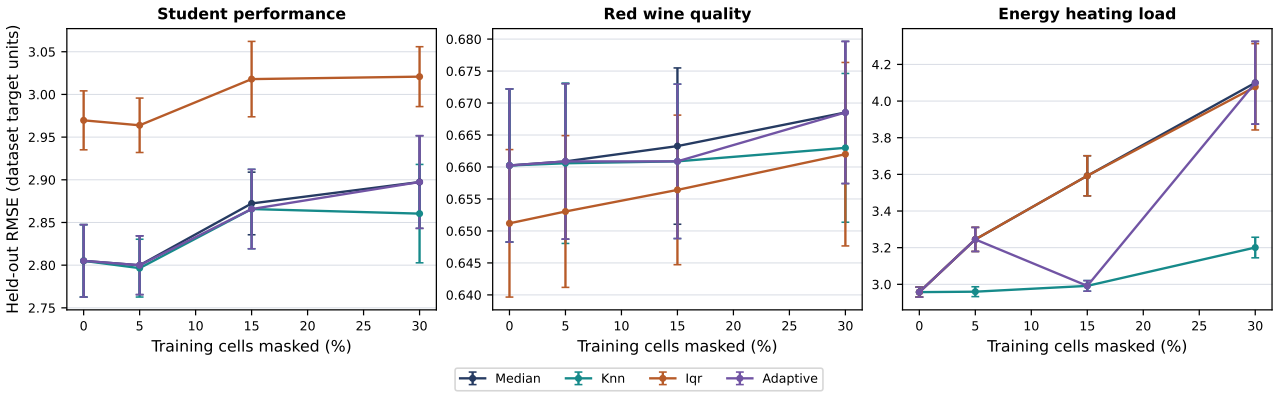


Fig. 2. Held-out Ridge RMSE under random training-feature masking. Points are means across five grouped splits and bars are standard errors. The three panels have different target units; compare strategies within each panel.

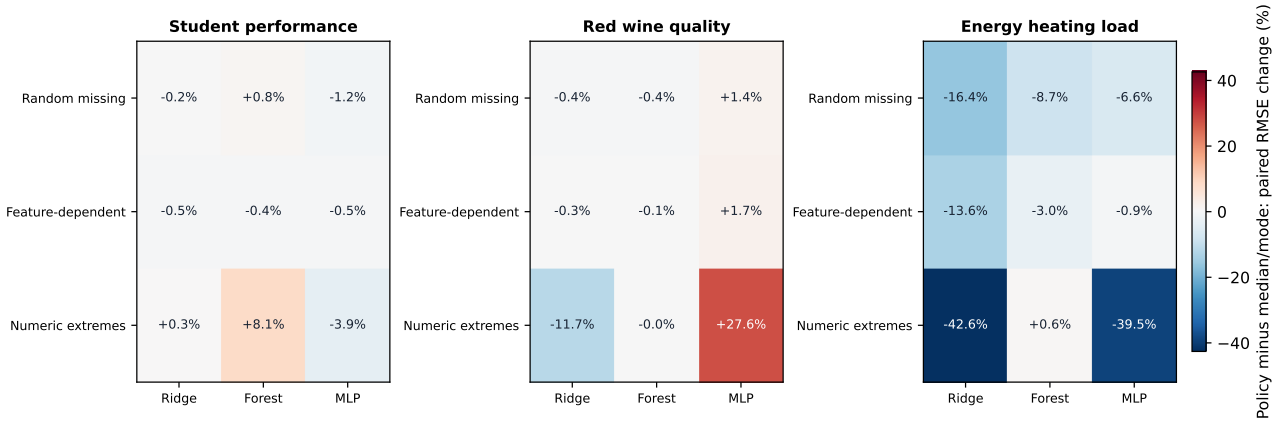


Fig. 3. Mean paired percentage change in held-out RMSE for the training-guided policy minus fixed median/mode at 15% training-feature corruption, across five matched splits. Blue (negative) favors the policy; red (positive) favors fixed median/mode. Each percentage is computed within a dataset and seed before averaging.

rather than a statistically indistinguishable alternative. When it chooses KNN or IQR, the paired score changes are attributable to that treatment under the fixed model. The five split repetitions describe variability within one task; their standard deviations in Table IV should not be misread as uncertainty over all possible datasets. A richer policy might incorporate column-level missingness, sample size after grouping, feature type, model family, or a training-only validation score. Evaluating such a policy would require a fresh prespecified protocol or nested validation, rather than revising thresholds after reading these held-out scores.

D. Runtime and application checks

The run-level timings show that richer cleaning is not free. Across positive-corruption conditions and all three models, the average fit-and-predict time per arm on the wine task was 0.550 seconds for median/mode, 0.679 seconds for KNN, 0.586 seconds for IQR, and 0.628 seconds for the policy. Student runs averaged 0.267, 0.290, 0.272, and 0.278 seconds in the same order; energy runs averaged 0.439, 0.461, 0.428, and 0.429 seconds. These times include model fitting and prediction, so they are descriptive comparisons on this machine rather than isolated imputer benchmarks. The complete fitting totals were 201.1 seconds for student, 437.7 for wine, and 309.9 for energy; the two latter studies were launched concurrently, so their total CPU-like durations are not a wall-clock project time. More expensive strategies need a predictive gain large enough to justify this added work.

The application-side profile supplies different evidence from model RMSE. The three source tables have no null cells; the wine table has 240 exact full-row duplicates, while the student and energy tables have none. The quality interface also flags numeric values beyond its IQR fences, but these are potential outliers, not labelled errors. A user can inspect columns, choose a target and unavailable features, run the matched experiment, view selected recipes, and export individual rows. The interactive student smoke run completed 32 fitted comparisons, and the separate command-line quick check produced 24 rows. Both use the same experiment function as the

full study. Optional training-only duplicate removal and deterministic category normalization are implemented for exploratory CSV comparisons; their effects are not mixed into the four-strategy results reported above.

E. Threats to validity and practical interpretation

Three modest public datasets, one set of fixed model hyperparameters, and five grouped random splits are a limited empirical basis. The wine target is ordinal though evaluated with regression loss. The energy data are simulated, and the extreme-value generator can change integer-coded features in ways not representative of a physical sensor. The feature-dependent mechanism is associated with an observed training driver but does not replicate an unknown real collection process. The clean held-out design answers a training-quality question, not a serving-data resilience question. No natural error labels are available to prove that an IQR flag is incorrect data. The neural architecture and iteration limit were fixed for comparability, and some MLP fits raised convergence warnings. Accordingly, the study supports a bounded conclusion: a cheap training-guided rule can help in particular corruption/model settings, but should be measured against simple fixed preprocessing on each task.

VII. Conclusion

DataSentry AI combines an inspectable CSV quality report, a local demonstration interface, and a reproducible paired experiment on three tabular regression tasks. Its 2,160 fitted arms show that training-feature corruption generally changes held-out error, with magnitude and direction that depend on dataset, mechanism, and model. The frozen training-guided policy beats fixed median/mode in 31 of 81 positive-corruption condition means, ties it in 36, and loses in 14. It is therefore useful as a transparent candidate, not a universally better cleaning rule. Synthetic extreme values on the energy task create large gains for Ridge and the neural comparator, while other conditions show meaningful harm from the same policy choice. The practical lesson is to report cleaning decisions and held-out effects together, preserve the split and fitting boundary, and retain negative outcomes. Future work should test naturally labelled errors, polluted serving inputs, more domains, and policy rules selected solely within nested training validation.

References

[1] E. Breck, N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, "Data validation for machine learning," in Proceedings of the 2nd SysML Conference, 2019. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2019/file/928f1160e52192e3e0017fb63ab65391-Paper.pdf

[2] M. Mahdavi, Z. Abedjan, R. C. Fernandez, S. Madden, M. Ouzzani, M. Stonebraker, and N. Tang, "Raha: A configuration-free error detection system," in Proceedings of the 2019 International Conference on Management of Data, 2019.

[3] P. Li, X. Rao, J. Blase, Y. Zhang, X. Chu, and C. Zhang, "Cleanml: A study for evaluating the impact of data cleaning on ml classification tasks," in 2021 IEEE 37th International Conference on Data Engineering, 2021.

[4] S. Krishnan, J. Wang, E. Wu, M. J. Franklin, and K. Goldberg, "Activeclean: Interactive data cleaning for statistical modeling," Proceedings of the VLDB Endowment, vol. 9, no. 12, pp. 948–959, 2016. [Online]. Available: https://www.vldb.org/pvldb/vol9/p948-krishnan.pdf

[5] S. Krishnan, M. J. Franklin, K. Goldberg, and E. Wu, "Boostclean: Automated error detection and repair for machine learning," 2017. [Online]. Available: https://arxiv.org/abs/1711.01299

[6] F. Neutatz, B. Chen, Y. Alkhatib, J. Ye, and Z. Abedjan, "Data cleaning and automl: Would an optimizer choose to clean?" Datenbank-Spektrum, vol. 22, pp. 121–130, 2022.

[7] S. Jäger and F. Biessmann, "From data imputation to data cleaning: Automated cleaning of tabular data improves downstream predictive performance," in Proceedings of the 27th International Conference on Artificial Intelligence and Statistics, ser. Proceedings of Machine Learning Research, vol. 238, 2024, pp. 3394–3402. [Online]. Available: https://proceedings.mlr.press/v238/jager24a.html

[8] D. Jarrett, B. C. Cebere, T. Liu, A. Curth, and M. van der Schaar, "Hyperimpute: Generalized iterative imputation with automatic model selection," in Proceedings of the 39th International Conference on Machine Learning, ser. Proceedings of Machine Learning Research, vol. 162, 2022, pp. 9916–9937. [Online]. Available: https://proceedings.mlr.press/v162/jarrett22a.html

[9] S. Mohammed, L. Budach, M. Feuerpfeil, N. Ihde, A. Nathansen, N. Noack, H. Patzlaff, F. Naumann, and H. Harmouch, "The effects of data quality on machine learning performance on tabular data," Information Systems, vol. 132, p. 102549, 2025.

[10] P. Cortez and A. Silva, "Using data mining to predict secondary school student performance," in Proceedings of the 5th Future Business Technology Conference, 2008, pp. 5–12. [Online]. Available: https://citeseerx.ist.psu.edu/document?doi=61d468d5254730bbecf822c6b60d7d6595d9889c

[11] P. Cortez, "Student performance dataset," UCI Machine Learning Repository, 2014.

[12] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," ACM Transactions on Knowledge Discovery from Data, vol. 6, no. 4, 2012.

[13] B. Bischl, G. Casalicchio, M. Feurer, P. Gijsbers, F. Hutter, M. Lang, R. G. Mantovani, J. N. van Rijn, and J. Vanschoren, "Openml benchmarking suites," in Advances in Neural Information Processing Systems, Datasets and Benchmarks Track, 2021. [Online]. Available: https://datasets-benchmarks-proceedings.neurips.cc/paper_files/paper/2021/hash/c7e1249ffc03eb9ded908c236bd1996d-Abstract-round2.html

[14] S. Schelter, T. Rukat, and F. Biessmann, "Jenga: A framework to study the impact of data errors on the predictions of machine learning models," in Proceedings of the 24th International Conference on Extending Database Technology, 2021. [Online]. Available: https://ssc.io/pdf/jenga.pdf

[15] N. Gupta, H. Patel, S. Afzal, N. Panwar, R. S. Mittal, S. Guttula, A. Jain, L. Nagalapatti, S. Mehta, S. Hans, P. Lohia, A. Aggarwal, and D. Saha, "Data quality toolkit: Automatic assessment of data quality and remediation for machine learning datasets," 2021. [Online]. Available: https://arxiv.org/abs/2108.05935

[16] T. Rekatsinas, X. Chu, I. F. Ilyas, and C. Ré, "Holoclean: Holistic data repairs with probabilistic inference," Proceedings of the VLDB Endowment, vol. 10, no. 11, pp. 1190–1201, 2017. [Online]. Available: https://www.vldb.org/pvldb/vol10/p1190-rekatsinas.pdf

[17] A. Lew, M. Agrawal, D. Sontag, and V. Mansinghka, "Pclean: Bayesian data cleaning at scale with domain-specific probabilistic programming," in Proceedings of the 24th International Conference on Artificial Intelligence and Statistics, ser. Proceedings of Machine Learning Research, vol. 130, 2021, pp. 1927–1935. [Online]. Available: https://proceedings.mlr.press/v130/lew21a.html

[18] K. Goyle, Q. Xie, and V. Goyle, "Dataassist: A machine learning approach to data cleaning and preparation," in Intelligent Systems and Applications, 2024, pp. 476–486.
[19] Z. Liu, Z. Zhou, and T. Rekatsinas, "Picket: Guarding against corrupted data in tabular data during learning and inference," 2021. [Online]. Available: https://arxiv.org/abs/2006.04730
[20] S. Eduardo, A. Nazabal, C. K. I. Williams, and C. Sutton, "Robust variational autoencoders for outlier detection and repair of mixed-type data," in Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics, ser. Proceedings of Machine Learning Research, vol. 108, 2020, pp. 4056–4066. [Online]. Available: https://proceedings.mlr.press/v108/eduardo20a.html
[21] D. B. Rubin, "Inference and missing data," Biometrika, vol. 63, no. 3, pp. 581–592, 1976.
[22] J. L. Schafer and J. W. Graham, "Missing data: Our view of the state of the art," Psychological Methods, vol. 7, no. 2, pp. 147–177, 2002.
[23] O. Troyanskaya, M. Cantor, G. Sherlock, P. Brown, T. Hastie, R. Tibshirani, D. Botstein, and R. B. Altman, "Missing value estimation methods for dna microarrays," Bioinformatics, vol. 17, no. 6, pp. 520–525, 2001.
[24] S. van Buuren and K. Groothuis-Oudshoorn, "mice: Multivariate imputation by chained equations in r," Journal of Statistical Software, vol. 45, no. 3, pp. 1–67, 2011.
[25] D. J. Stekhoven and P. Bühlmann, "Missforest: Non-parametric missing value imputation for mixed-type data," Bioinformatics, vol. 28, no. 1, pp. 112–118, 2012.
[26] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "Lof: Identifying density-based local outliers," in Proceedings of the ACM SIGMOD International Conference on Management of Data, 2000, pp. 93–104.
[27] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in 2008 Eighth IEEE International Conference on Data Mining, 2008, pp. 413–422.
[28] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," ACM Computing Surveys, vol. 41, no. 3, pp. 15:1–15:58, 2009.
[29] L. Breiman, "Random forests," Machine Learning, vol. 45, pp. 5–32, 2001.
[30] Y. Gorishniy, I. Rubachev, V. Khrulkov, and A. Babenko, "Revisiting deep learning models for tabular data," in Advances in Neural Information Processing Systems, 2021. [Online]. Available: https://papers.nips.cc/paper/2021/file/9d86d83f925f2149e9edb0ac3b49229c-Paper.pdf
[31] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in Advances in Neural Information Processing Systems, 2022.
[32] D. McElfresh, S. Khandagale, J. Valverde, V. P. C, B. Feuer, C. Hegde, G. Ramakrishnan, M. Goldblum, and C. White, "When do neural nets outperform boosted trees on tabular data?" in Advances in Neural Information Processing Systems, 2023. [Online]. Available: https://papers.nips.cc/paper/2023/file/f06d5ebd4ff40b40dd97e30cee632123-Paper-Datasets_and_Benchmarks.pdf
[33] W. McKinney, "Data structures for statistical computing in python," in Proceedings of the 9th Python in Science Conference, 2010, pp. 51–56.
[34] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://www.jmlr.org/papers/v12/pedregosa11a.html